VIETNAM NATIONAL UNIVERSITY OF HO CHI MINH CITY
THE INTERNATIONAL UNIVERSITY
SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

Flood Situation Monitoring and Reporting System

By
Do Nguyen Chuong - ITITIU22021

A thesis submitted to the School of Computer Science and Engineering
in partial fulfillment of the requirements for the degree of
Bachelor of Computer Science

Ho Chi Minh City, Vietnam
2026

Flood Situation Monitoring and Reporting System

APPROVED BY:

Le Duy Tan, Ph.D., supervisor

THESIS COMMITTEE

Le Duy Tan, Ph.D

.....

Acknowledgments

I would like to thank my thesis supervisor, Dr. Le Duy Tan, for his guidance throughout this work. His comments helped me narrow the topic, connect the system design with the actual source code, and keep the report focused when the project started to become too wide. I am especially grateful for the time he spent reviewing my ideas and pointing out where the work still needed to be clearer.

I also want to thank the lecturers at the School of Computer Science and Engineering. The courses and discussions I had there gave me the background I needed to build VietFlood across backend services, a mobile app, a web dashboard, and deployment tools.

I am thankful to the staff at International University as well. Their work behind the scenes made my study process smoother than it would have been otherwise.

My classmates and colleagues also helped more than they may realize. Some helped me talk through technical problems. Some simply listened when the project felt messy. Those small conversations made the work easier to continue.

Finally, I would like to thank my family for their patience and support during my studies. Their trust in me gave me room to finish this thesis.

This thesis carries the help of many people. I am grateful to all of them.

Table of Contents

List of Tables

List of Figures

List of Abbreviations

API	Application Programming Interface
CI/CD	Continuous Integration and Continuous Deployment
ORM	Object-Relational Mapping
RBAC	Role-Based Access Control
HTTP	Hypertext Transfer Protocol
UI/UX	User Interface / User Experience
CDN	Content Delivery Network
PUB/SUB	Publish–Subscribe
URL	Uniform Resource Locator
JSON	JavaScript Object Notation
JWT	JSON Web Token
TTL	Time To Live
WS	WebSocket

Abstract

Flood response depends on small pieces of local information becoming useful at the right time. In many cases, the first sign of a flood problem is not an official record but a photo, a short description, or a shared location from a person already near the affected area. When those signals stay inside separate chat groups or informal channels, the information is hard to verify, search, and follow up. This thesis presents VietFlood, a prototype for a Flood Situation Monitoring and Reporting System that organizes citizen flood reports for relief awareness and administrator review.

VietFlood is implemented as a multi-platform prototype. The backend uses NestJS with an API gateway, an authentication service, a reports service, and a Socket.IO tracking gateway. RabbitMQ supports service-to-service communication, PostgreSQL stores users, refresh tokens, and flood reports through TypeORM, Redis supports cached report lookup, and Cloudinary stores uploaded evidence files. Shared infrastructure code is kept in `vietflood_common`, which provides reusable logging, Redis, and Cloudinary helpers. The mobile application is built with Expo React Native for citizen and relief workflows. The web dashboard is built with Next.js, React, TypeScript, and Tailwind CSS for administrators.

The prototype covers the main workflow from report creation to review. Citizens can register, sign in, create flood reports, provide administrative address fields and coordinates, set urgency and severity, and upload photo evidence. Relief users can open broader report views and map-related screens. Administrators can use the web dashboard to inspect reporter information, search and filter reports, preview evidence, manage users, and update report status using pending, verified, resolved, and rejected states. VietFlood also includes a live tracking path in which connected clients send location updates through Socket.IO after latitude and longitude validation at the gateway.

The work focuses on system design and implementation rather than flood prediction or official emergency dispatch. The final prototype demonstrates how mobile reporting, role-based access control, evidence storage, real-time location events, shared backend utilities, and container-based deployment can be combined in a Vietnam-focused flood reporting system. Although it remains a prototype, VietFlood provides a working foundation for collecting field reports and making them easier to review.

Keywords: VietFlood; flood reporting; relief coordination; real-time tracking; microservice architecture; NestJS; React Native; Next.js; RabbitMQ; PostgreSQL; Redis; Cloudinary.

Chapter 1
Introduction

1.1 The necessity of flood reporting and relief coordination by information technology

1.1.1 What is flood reporting and relief coordination by technology?

Flood information is often noticed first by people who are already in or near the affected area. A resident may see a road becoming unsafe. A volunteer may know that one route is still open while another route is blocked. A nearby user may only have time to send a short description, a photo, or a location. These small reports matter, but they lose value when they are separated from the details needed for review.

In this thesis, technology-supported flood reporting means converting those field observations into structured records that can be stored, searched, and checked later. The idea is related to citizen-generated geographic and crisis information, where local observations can help describe a situation when they are organized for processing and decision support [1, 2]. For VietFlood, a report is treated as more than a message. It should carry the flood category, description, province, ward, address, coordinates, urgency, severity, evidence files, reporter identity, and review status in one record.

VietFlood is developed for this reporting need. It is a multi-platform prototype for a Flood Situation Monitoring and Reporting System in Vietnam. The system brings together a mobile application for citizens and relief users, a web dashboard for administrators, a NestJS backend, and a shared TypeScript infrastructure package [3, 4]. The project does not attempt to forecast floods. Its focus is the later stage, when people have already observed a flood situation and the information needs to be submitted, stored, and reviewed.

The backend project, `VietFlood`, is organized around four main runtime parts: an API gateway, an authentication service, a reports service, and a Socket.IO tracking gateway. The API gateway is the public entry point for HTTP and WebSocket clients. The authentication service manages registration, sign-in, profile loading, refresh tokens, logout, and user management. The reports service stores report data, evidence metadata, status changes, and cache updates.

The project also includes `vietflood_common`, a shared package used by backend services. It contains common logging, Redis, and Cloudinary helpers. This separation is useful because logging, cache access, and media uploads should not behave differently from one service to another. By keeping this infrastructure code in one package, the backend can reuse the same behavior instead of copying similar helper code into each service.

1.1.2 Some approaches to supporting flood reporting and relief coordination using technology

VietFlood uses several practical approaches to support flood reporting and review. The first is a mobile-first reporting flow. Flood reports are usually created in the field, so the citizen and relief workflows are implemented in an Expo React Native application with TypeScript [5, 6, 4]. The mobile app supports authentication, profile screens, report creation, image picking, location-aware screens, relief views, maps, user overview pages, role-aware navigation, and Vietnamese or English interface text. It also uses the Vietnam Provinces Open API so address fields can be selected more consistently [7].

The second approach is separating responsibilities by role. VietFlood uses citizen, relief, and admin roles. Citizens submit reports and view their own records. Relief users can view broader report information and map-related screens for field awareness. Administrators work from the web dashboard, where they can inspect evidence, search and filter reports, view reporter details, update report status, and manage users. The dashboard checks the current profile before showing admin content, so role control is not treated as only a visual decision in the interface.

The third approach is evidence handling through external media storage. A flood report is easier to review when a photo is attached, but storing large image files directly in PostgreSQL is not suitable for this prototype. VietFlood receives uploaded files through the API gateway, sends them to Cloudinary under the `vietflood/reports` folder, and stores the returned URL, public ID, and resource type with the report data [8]. This allows the dashboard to preview evidence while still keeping enough metadata to delete remote files when a report is removed.

The fourth approach is a small microservice backend. Microservice architecture is commonly described as splitting a system into services around separate capabilities instead of building one large application [9, 10, 11]. In VietFlood, the API gateway communicates with the authentication and reports services through RabbitMQ message patterns [12, 13]. The authentication service consumes messages from `auth_queue`, while the reports service consumes messages from `reports_queue`. PostgreSQL stores persistent data through TypeORM, and Redis is used for cached report lookup data [14, 15, 16]. This design is more complex than a single server, but it makes the responsibility of each service easier to explain and maintain.

The fifth approach is live location broadcasting. VietFlood includes a Socket.IO tracking path where connected clients can send location updates [17]. The gateway validates latitude and longitude before broadcasting the update to other clients. Invalid payloads return an error event. This is not a complete emergency dispatch feature, but it demonstrates how live movement information can be added to the reporting system.

1.2 Problem Statement

Flood reports are less useful when the important pieces are scattered. A photo without a location may not tell an administrator where to look. A coordinate without a description may not explain the risk. A report without a status makes it hard to know whether someone has reviewed it. These problems become more serious when many reports arrive during heavy rain.

The main problem addressed in this thesis is:

How can VietFlood collect flood reports from citizens, store the report data

and evidence safely, and present the reports in a form that relief users and administrators can review?

This problem includes several design concerns. The report form must collect enough information to be useful while still being quick enough for a mobile user. Evidence files must remain connected to the report record after upload. Report status must be visible so that pending, verified, resolved, and rejected cases can be separated. The backend must support both the mobile application and the web dashboard without forcing each client to duplicate the same business rules.

The thesis also defines what VietFlood does not solve. It does not provide flood prediction, rescue routing, satellite image processing, SMS warning delivery, official dispatch integration, offline-first synchronization, or formal GIS analysis. Those features would require different data sources, operational partners, and safety validation. The scope here is the prototype that was designed and implemented: account handling, report submission, evidence upload, role-based review, live location events, caching, and container-based deployment.

1.3 Scope And Objectives

The scope of VietFlood is divided into four project modules. The first module is the backend service group, `VietFlood`. It contains the API gateway, authentication service, reports service, and tracking gateway. The gateway exposes authentication and report routes, receives evidence uploads, and handles Socket.IO events. The authentication service owns user accounts and tokens. The reports service owns report records, evidence metadata, status updates, and report cache invalidation.

The second module is `vietflood_common`. This shared package provides backend infrastructure that is reused across services. It includes structured logging, Redis access, and Cloudinary upload or deletion helpers. The logger can include service name, trace context, user ID, role, request path, and method when request context is available.

The third module is `VietFlood_mobile`. It is the Expo React Native application for citizen and relief workflows. It includes login, registration, profile screens, report creation, image attachment, citizen report lists, relief report details, map screens, user overview screens, settings, and localization support. The app also normalizes backend response data because report coordinates and field names may appear in different forms such as `lat`, `lng`, latitude, longitude, snake case, and camel case.

The fourth module is `Vietflood_web`. It is a Next.js dashboard for administrator monitoring. The dashboard signs in through the backend, stores access and refresh tokens, refreshes sessions when required, loads the current profile, blocks non-admin accounts from dashboard use, and displays reports with search, status filters, evidence previews, reporter information, and status update actions.

The deployment scope covers Docker Compose for the backend services, Redis, and RabbitMQ [18]. The production-oriented setup uses Docker Hub images and Azure App Service [19]. GitHub Actions builds and tags service images, pushes them to Docker Hub, configures the Azure App Service deployment, and restarts the application [20]. Environment variables are used for database URLs, RabbitMQ credentials, Redis password, JWT secrets, refresh-token secrets, Cloudinary credentials, and admin seed values.

1.3.1 Goals of VietFlood

The main goal of VietFlood is to deliver a working flood reporting prototype for Vietnam. The prototype should allow citizens to submit reports from mobile devices, preserve photo evidence, enforce role-based access, and give administrators a readable interface for report follow-up.

The specific goals are:

1. Build a NestJS backend that separates public access, authentication, report handling, and live tracking.
2. Use RabbitMQ message patterns for communication between the API gateway and internal services.
3. Store user accounts, refresh tokens, and flood reports in PostgreSQL through TypeORM.
4. Protect citizen, relief, and admin workflows with JWT access tokens, refresh tokens, and role guards.
5. Support report creation with category, description, address fields, coordinates, urgency, severity, and photo evidence.
6. Upload evidence to Cloudinary and keep the returned URL and public ID with the report record.
7. Use Redis for report lookup caching and clear stale cached data after report changes.
8. Provide an Expo React Native mobile app for citizen reporting and relief-oriented views.
9. Provide a Next.js web dashboard for admin login, report search, evidence review, user management, and status updates.
10. Prepare Docker Compose and CI/CD configuration for local and production-style backend deployment.

1.3.2 System Actors and Basic Functions

VietFlood uses three roles: citizen, relief, and admin. The role model is intentionally simple because the prototype only needs enough separation to protect personal reports, field awareness screens, and administrator review actions.

Citizens use the mobile app to register, sign in, manage their profile, create flood reports, attach evidence, provide location information, and view reports linked to their own account. This actor represents people who are close enough to the flood situation to submit first-hand information.

Relief users use the mobile app for wider awareness. They can inspect report lists, open report details, and use map-related views. In this prototype, relief users do not verify or reject reports. Their role is focused on reading and field awareness rather than system administration.

Administrators use the web dashboard. They can sign in, load reports, search by report or reporter information, filter by status, preview evidence, inspect reporter details,

update report status, and manage user records. The dashboard checks the signed-in profile before allowing access to admin pages.

All three roles connect through the same backend entry point. Mobile and web clients call the API gateway, and the gateway decides whether the request should go to authentication, report handling, evidence upload, or live tracking. This keeps client applications focused on user interaction while backend services handle system rules.

1.3.3 Thesis Structure

This thesis is organized into six chapters. Chapter 1 introduces the motivation, problem statement, scope, objectives, actors, and overall structure of VietFlood. Chapter 2 reviews background and related work on flood reporting, citizen-generated information, role-based review, microservices, message queues, media evidence, caching, real-time communication, and deployment.

Chapter 3 presents the methodology, including the project modules, actors, architecture, database design, use cases, workflows, deployment method, and evaluation plan. Chapter 4 describes the implementation and result of the backend, mobile application, web dashboard, shared library, and CI/CD setup. Chapter 5 evaluates the prototype and discusses current limitations. Chapter 6 concludes the thesis and outlines future work.

Chapter 2
Background and Related Works

2.1 Background

Flood reporting is a practical information problem before it becomes a software problem. A person near a flooded road may know what is happening before any official record is created, but that first message is often incomplete. It may include a photo but no ward name, a description but no coordinates, or a location pin without enough context to judge the situation. VietFlood is built around this gap. The system tries to turn quick local observations into structured reports that can be stored, searched, and reviewed later. This direction is consistent with the use of volunteered and citizen-generated information in crisis response, where local observations become more useful when they are organized for processing [1, 2].

The project does not aim to replace forecasting systems, hydrological models, or official emergency command centers. Its role is narrower and more concrete. VietFlood supports report submission from a mobile device, evidence upload, report storage, status tracking, relief awareness screens, and administrator review through a web dashboard. This scope is important because a flood management platform can become very broad. The thesis focuses on the part that was designed and implemented: moving field information from a citizen report into a backend record that an authorized user can inspect.

2.1.1 Flood Reports as Structured Field Records

An informal flood message can be fast, but it is not always easy to reuse. A chat message may be buried after a few minutes. A photo may be forwarded without its original location. A phone call may help one person but leave no shared record for others. VietFlood treats each citizen submission as a structured field record instead of a loose message.

The report entity in VietFlood stores the category, description, image links, evidence metadata, province, ward, address line, latitude, longitude, urgency flag, severity value, status, timestamps, and the account that created the report. These fields answer the questions that usually come first during review: what happened, where it happened, who reported it, how serious it appears, and what evidence supports the report.

The report status model is intentionally small. A report can be `pending`, `verified`, `resolved`, or `rejected`. A new submission begins as pending. An administrator can mark it as verified when the information is considered reliable, resolved when the case is no longer active, or rejected when the report is unsuitable or incorrect. Keeping rejected reports separate is useful because it preserves an audit trail instead of simply deleting every bad report.

2.1.2 The Role of Mobile Reporting

Mobile reporting is central to VietFlood because most observations are made away from a desk. A citizen may be standing near a flooded street, using a phone connection, and trying to submit a report quickly. For that reason, the citizen and relief-side application is built with Expo React Native [5, 6]. The same mobile codebase supports authentication,

profile pages, report creation, image selection, report lists, relief-oriented screens, maps, and role-aware navigation.

Figure 2.1 presents the normal life of a report in the prototype. A citizen creates the report on the mobile app. The request reaches the API gateway, evidence files are uploaded, the reports service stores the record, and an administrator can later inspect the case and update its status. The diagram is kept simple because Chapter 2 explains the background idea. The detailed service architecture is discussed later in the methodology chapter.

The mobile form also supports location consistency. VietFlood uses the Vietnam Provinces Open API so users can select province, district, and ward information from known administrative divisions instead of typing every field freely [7]. This reduces spelling variation and makes the data easier to filter. Coordinates are also part of the report model, and the mobile code handles different field names such as `lat`, `lng`, `latitude`, and `longitude` when normalizing backend responses.

2.1.3 The Role of Web-Based Review

The web application has a different purpose from the mobile app. It is not designed for quick field submission. It is designed for administrator review, where a user may need to scan many reports, inspect evidence, search by report or reporter information, and update status. VietFlood's web dashboard is built with Next.js, React, TypeScript, and Tailwind CSS [21, 22, 4, 23].

The dashboard receives report data through the same backend gateway used by the mobile app. It displays report status, category, severity, address, description, evidence thumbnails, reporter information, and creation time. Search and filtering are important because a report list becomes difficult to use when many cases are pending at the same time. Administrators can focus on a status group, a reporter, or a location-related keyword instead of manually reading every card.

Access control is part of the review design. A citizen account should not enter the administrator dashboard simply by opening a page URL. The web client checks the signed-in profile before showing admin pages, and backend routes use JWT and role-based guards where protected access is needed. This keeps interface behavior and backend permission checks aligned.

2.2 Related Work and Technical Foundations

The technical foundation of VietFlood comes from several common patterns rather than one copied application. The project combines mobile field reporting, role-based access, a small microservice backend, message queues, external media storage, caching, real-time socket communication, and containerized deployment. Each part solves a specific problem in the reporting workflow.

2.2.1 Informal Reporting Channels

In real communities, flood information often begins in informal channels such as phone calls, messaging apps, social media posts, or volunteer groups. These channels are fast because people already know how to use them. A resident can send a photo in seconds. A volunteer can forward a warning to a group. For early awareness, that speed is useful.

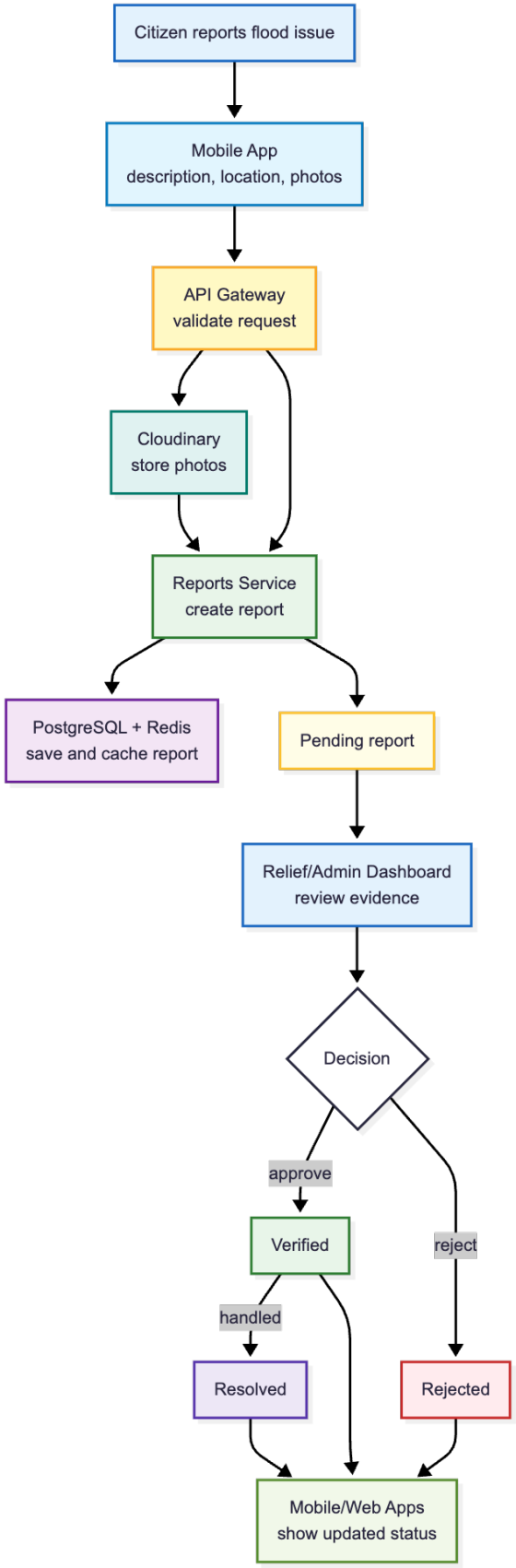

Figure 2.1: Lifecycle of a VietFlood citizen report from mobile submission to review status.

The weakness appears after the first wave of messages. Informal channels do not naturally create a clean report database. Duplicate reports, missing addresses, old photos, unclear status, and inconsistent reporter information can all appear in the same conversation. It is also hard to know whether a message has been reviewed, verified, resolved, or rejected.

VietFlood keeps the citizen reporting idea but adds structure. The mobile app asks for report details, location information, urgency, severity, and evidence. The backend stores the submission as a report record, and the dashboard gives administrators a shared place to search and review it. The system therefore does not replace human judgment; it gives that judgment a clearer record to work with.

2.2.2 Centralized Web Systems

One simple way to build a reporting system is to use a centralized web application. A single backend can handle login, users, reports, files, status updates, and admin pages. This approach is easy to understand and can be suitable for a small application. It also reduces the number of moving parts during development.

VietFlood uses a small microservice design instead. This choice was made because the project needs clear boundaries between public access, authentication, report handling, media upload, caching, and live tracking. Microservice architecture is often described as a way to separate a system around business or technical capabilities, while accepting extra operational cost [9, 11, 10, 24]. VietFlood keeps the split modest: an API gateway, an authentication service, a reports service, Redis, RabbitMQ, PostgreSQL, and Cloudinary integration.

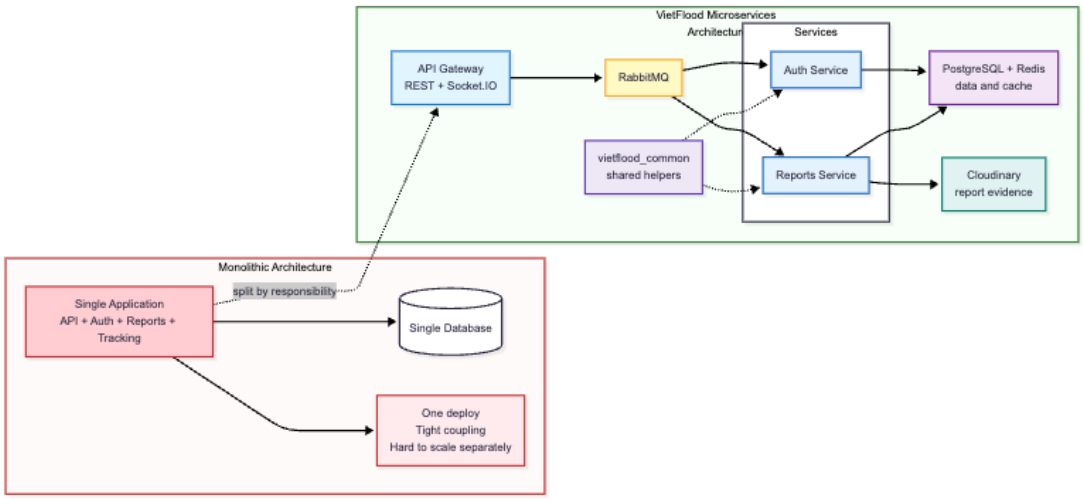

Figure 2.2: Comparison between a monolithic backend and the VietFlood microservices structure.

Figure 2.2 shows the difference between a single backend and the VietFlood structure. In VietFlood, the API gateway handles the public interface, the authentication service owns identity and tokens, and the reports service owns flood reports and evidence metadata. This division is not meant to make the prototype larger for its own sake. It is meant to keep the responsibilities visible.

2.2.3 API Gateway Pattern

The API gateway is the public entrance to the VietFlood backend. Mobile and web clients do not call the authentication service or reports service directly. They call the gateway, and the gateway forwards the request to the correct internal service. This follows the common gateway pattern, where a client-facing component hides internal service structure and provides one stable API surface [24, 3].

In VietFlood, the gateway handles authentication routes, report routes, multipart evidence upload, and Socket.IO tracking events. For report creation, it receives uploaded files through the `files` field, sends the file buffers to Cloudinary, and forwards the report payload and evidence metadata to the reports service. This keeps file upload concerns close to the public HTTP layer while keeping report persistence inside the reports service.

The gateway also hosts the live tracking entry point. A connected client can send a `send-location` event. The gateway checks latitude and longitude before broadcasting a `receive-location` event. Invalid payloads return `location-error`, and disconnected sockets are announced through `user-disconnected`. These events are small, but they show how real-time data can be added without changing the normal report API.

2.2.4 Message Queues Between Services

RabbitMQ is used for communication between the API gateway and internal backend services [12, 13]. The gateway sends authentication-related messages to the auth queue and report-related messages to the reports queue. The corresponding service consumes the message, performs the work, and returns the result.

This design makes service boundaries clearer. For example, the gateway receives a sign-in request, but password checking and token creation belong to the authentication service. The gateway receives a create-report request, but report persistence belongs to the reports service. The gateway coordinates the request; it does not own all business logic.

Message-based communication also changes how errors are handled. A service may be unavailable or slow, so the gateway needs controlled timeouts and predictable responses. This is more work than a direct function call, but it reflects the realities of distributed systems and gives the project a useful technical foundation.

2.2.5 Authentication and Role-Based Access

VietFlood uses password-based authentication, JWT access tokens, refresh tokens, and role checks. Passwords are hashed with bcrypt before storage [25]. After sign-in, the system returns an access token and refresh token. The access token follows the JWT model, where signed claims can be checked by protected routes [26].

The project uses three roles: citizen, relief, and admin. Citizens create reports and view reports linked to their own account. Relief users use mobile screens for broader report awareness and map-related viewing. Administrators use the web dashboard to inspect reports, update status, and manage users. This separation matters because report review and user management should not be available to every signed-in user.

The mobile application uses role-aware navigation so users see screens that match their role. The web dashboard checks the current profile before allowing admin pages to appear. The backend protects sensitive operations with authentication and role guards. Together, these layers keep access control from depending only on the user interface.

2.2.6 Media Evidence and Cloud Storage

Photo evidence can make a flood report easier to judge. A description can say that a street is flooded, but an image can show water level, road blockage, damage, or surrounding conditions. VietFlood stores uploaded evidence in Cloudinary instead of storing raw image files directly in PostgreSQL [8].

The upload path begins at the API gateway. The gateway receives file buffers from the mobile app, uploads them to the `vietflood/reports` folder in Cloudinary, and passes the returned URL, public ID, and resource type to the reports service. The reports service stores this evidence metadata with the report. The URL supports preview and display. The public ID supports cleanup when a report is removed.

This design separates media storage from report storage. PostgreSQL keeps structured report data, while Cloudinary handles media files and delivery. Shared Cloudinary helpers in `vietflood_common` keep upload and delete behavior reusable across services.

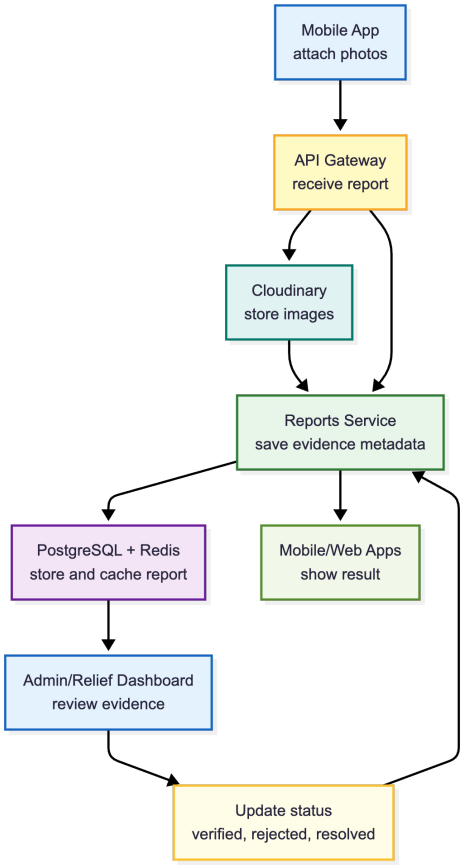

Figure 2.3: Evidence upload and review flow in VietFlood.

Figure 2.3 shows the technical path of evidence upload. The next figure pairs that system flow with screenshots from the actual application so the reader can connect the architecture to the user experience.

2.2.7 Database and Cache

PostgreSQL is the main persistent database for VietFlood. The backend uses TypeORM to map application entities to database tables [14]. Reports store both ordinary relational fields, such as status and coordinates, and JSON evidence metadata. PostgreSQL supports JSON data when flexible structured fields are needed inside a relational database

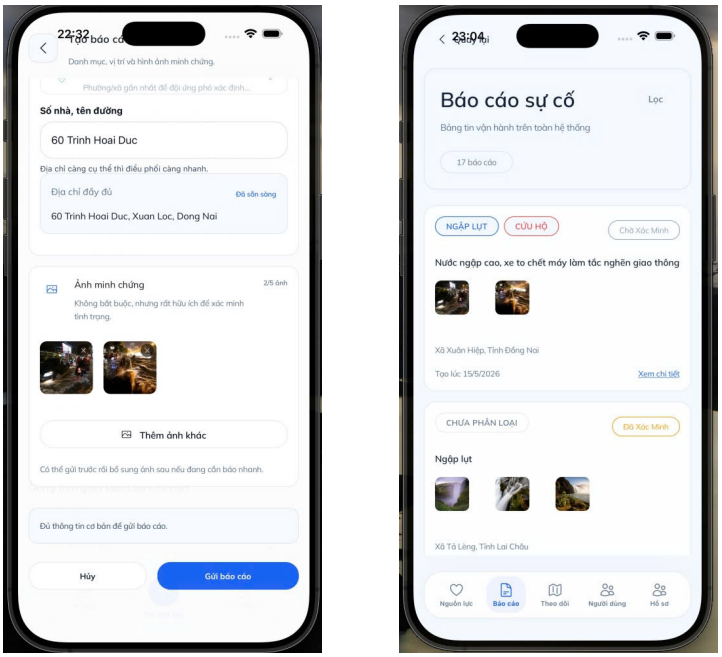

Figure 2.4: Mobile evidence upload and dashboard evidence review in VietFlood.

[15].

Redis is used as a focused cache rather than a replacement for the database [16]. The reports service can cache report lookup data and clear stale entries when a report changes. PostgreSQL remains the source of truth, while Redis helps reduce repeated reads for data that may be requested multiple times.

The cache scope is deliberately limited. A broad cache layer can become difficult to reason about if invalidation is unclear. VietFlood uses caching where the key pattern and update behavior are understandable, which fits the size of the prototype.

2.2.8 Shared Infrastructure Package

The backend services reuse common infrastructure through the `vietflood_common` package. This package exports the Redis module and service, Cloudinary module and service, and a structured logger. The logger can include timestamp, log level, service name, trace context, user ID, role, request path, and HTTP method when that context is available.

The shared package reduces repeated code across the gateway, authentication service, and reports service. More importantly, it keeps infrastructure behavior consistent. A Redis connection should be configured the same way across services. A Cloudinary upload should return evidence metadata in the same shape. A log message should carry enough context to trace work through the backend.

2.2.9 Containerized Deployment

VietFlood uses Docker Compose to run the backend services together with Redis and RabbitMQ [18]. The local setup builds service images from the repository, while the production-oriented configuration uses Docker Hub images for the gateway, auth service, and reports service. Environment variables provide database URLs, Redis settings, RabbitMQ credentials, JWT secrets, refresh-token secrets, Cloudinary credentials, and admin seed values.

The project also includes a GitHub Actions deployment workflow. The workflow builds

service images, tags them, pushes them to Docker Hub, configures Azure App Service, and restarts the deployed application [20, 19]. This is aligned with continuous delivery practices, where builds and deployments should be repeatable instead of depending on manual steps [27, 28].

2.3 Comparison of Design Approaches

VietFlood can be compared with two simpler approaches. The first is informal reporting through chat, phone calls, and social media. This is fast and familiar, but it does not produce a searchable report database or a clear status history. The second is a single-server reporting website. This is easier to develop, but authentication, reports, evidence, caching, and live tracking can become tightly connected as the project grows.

The VietFlood approach sits between those two options. It is more structured than informal reporting and more modular than a single backend, but it is still small enough to explain clearly. The API gateway owns the public interface. The authentication service owns accounts and tokens. The reports service owns flood report data and evidence metadata. Redis, PostgreSQL, RabbitMQ, Cloudinary, and Socket.IO each support a specific infrastructure need.

This design adds engineering effort. There are more services to configure, more environment variables to manage, and more failure points to consider. The benefit is that each responsibility is visible. For a thesis prototype, that tradeoff is useful because it demonstrates not only a working application, but also the architectural reasoning behind the application.

Chapter 3
Methodology

This chapter describes the method used to design and implement VietFlood. The work followed a software-engineering approach rather than an experimental survey method. First, the main actors and reporting workflow were defined. Second, the report data model was shaped around location, evidence, owner, urgency, severity, and review status. Third, the system was divided into backend services, shared infrastructure utilities, a mobile application, and a web dashboard. Finally, the main workflows were checked through source-level review, local deployment, and end-to-end use of the reporting path. This follows the architectural view that system structure should be guided by responsibilities and quality needs, not only by the chosen technology stack [24].

3.1 Requirement Analysis

The requirement analysis started from the central workflow of the project: a person observes a flood-related situation, submits a report from a mobile device, and an authorized reviewer later inspects the report. VietFlood does not attempt to solve the whole disaster-response process. The method focuses on the part that can be implemented and tested in the prototype: account handling, report submission, evidence upload, report review, and status tracking.

3.1.1 Functional Requirements

The functional requirements were grouped by the three roles supported by the system.

- **Citizen users** can register, sign in, update their profile, create flood reports, provide address and coordinate information, attach photo evidence, and view reports linked to their own account.
- **Relief users** can view broader report lists, open report details, inspect map-related screens, and use report information for field awareness. They do not verify or reject reports in this prototype.
- **Admin users** can use the web dashboard to view all reports, search and filter records, inspect reporter information, preview evidence, update report status, and manage user information.

The report object is the center of the system. Each report needs category data, a written description, administrative address fields, coordinates, evidence metadata, a reporter, a severity value, an urgency flag, timestamps, and a review status. The status values are kept simple: `pending`, `verified`, `resolved`, and `rejected`. This small set is enough for the prototype and avoids a review process that would be too complex for the current scope.

3.1.2 Non-Functional Requirements

The non-functional requirements came from the way the system is expected to be used. The backend should expose one public entry point for mobile and web clients. Authentication and report handling should be separated so identity logic does not become mixed with report storage and evidence handling. Uploaded images should be stored outside the relational database, while report records and evidence metadata remain searchable. The backend should also be deployable through repeatable configuration rather than manual server setup.

These requirements led to four main design decisions. VietFlood uses an API gateway in front of internal services. RabbitMQ carries messages between the gateway, authentication service, and reports service. Cloudinary stores uploaded evidence files, while PostgreSQL stores report records and evidence metadata. Docker Compose is used to run the backend services together with Redis and RabbitMQ in a repeatable environment.

3.2 Actor Analysis

VietFlood has three main actors: citizen, relief, and administrator. The actors were chosen to match the real responsibilities in the prototype. Citizens submit reports, relief users read field information, and administrators review and manage the submitted records.

3.2.1 Citizen Actor

The citizen actor represents a person who notices a flood-related incident and submits the first structured record. The citizen workflow begins with registration or sign-in. After authentication, the user can create a report by selecting a category, writing a description, choosing location information, setting urgency and severity, and attaching optional photos. The mobile app can also help with device location and Vietnam administrative division selection.

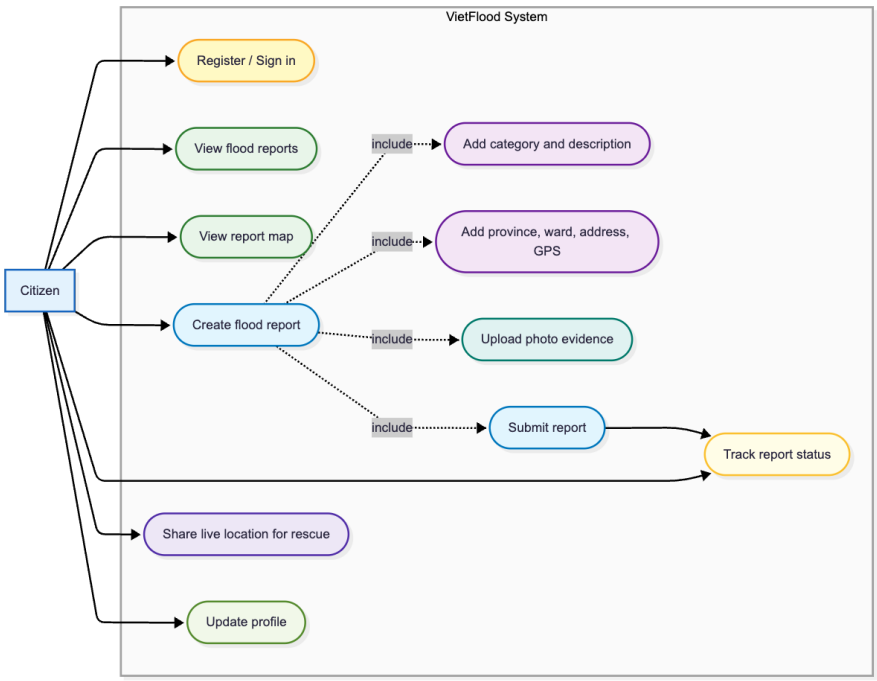

Figure 3.1: Use case diagram for the citizen actor in VietFlood.

3.2.2 Relief Actor

The relief actor represents users who need situational awareness from the submitted reports. In the mobile app, this role can use report lists, report detail screens, and map-related views. The relief role is deliberately narrower than the admin role. Relief users can inspect information for field awareness, but they do not verify, reject, or administratively close reports in this thesis version.

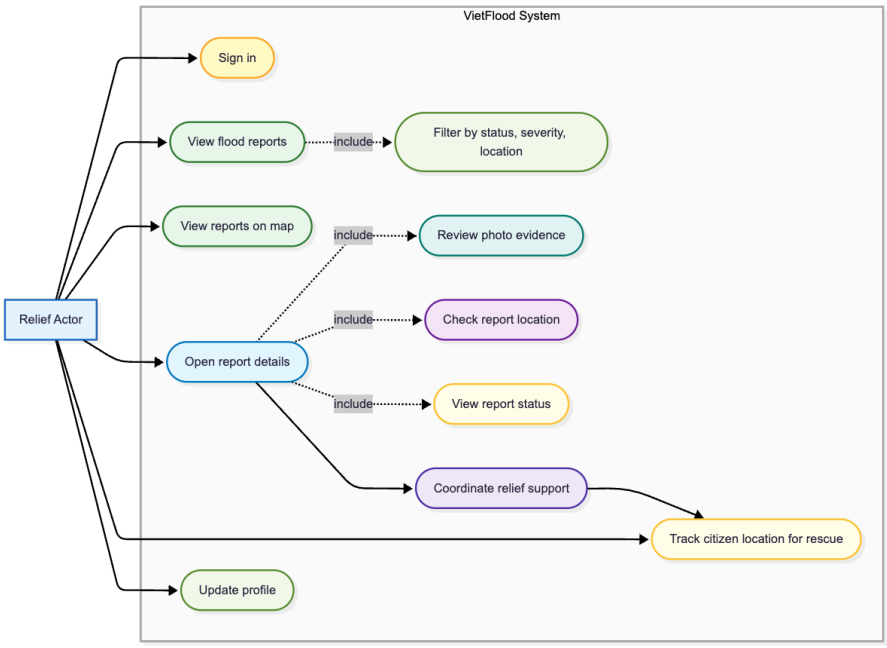

Figure 3.2: Use case diagram for the relief actor in VietFlood.

3.2.3 Administrator Actor

The administrator actor works through the web dashboard. The dashboard loads report data through the API gateway and displays report content together with reporter details, status, evidence previews, and filtering controls. Admin users are responsible for changing report status and managing user records. The web client checks the signed-in profile before showing the dashboard so a non-admin account cannot simply open the admin pages.

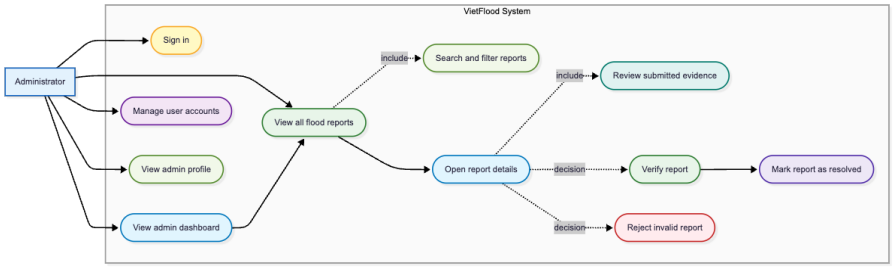

Figure 3.3: Use case diagram for the administrator actor in VietFlood.

3.3 Materials and Development Tools

VietFlood was developed from four source-code modules. The backend module contains the NestJS API gateway and microservices [3, 12]. The shared module contains reusable

logging, Redis, and Cloudinary utilities. The mobile module contains the Expo React Native application [5, 6]. The web module contains the Next.js administrator dashboard [21]. The main tools are summarized in Table 3.1.

Table 3.1: Main materials and tools used in VietFlood development.

Area	**Tools and purpose**
Backend	NestJS, TypeScript, TypeORM, PostgreSQL, RabbitMQ, Redis, Socket.IO, JWT, bcrypt.
Shared package	`vietflood_common` for structured logging, Redis client setup, and Cloudinary file operations.
Mobile client	Expo, React Native, TypeScript, React Navigation, Redux Toolkit, image picking, secure token storage, device location, and Vietnam Provinces Open API.
Web dashboard	Next.js, React, TypeScript, Tailwind CSS, protected admin login, token refresh, report overview, search, filtering, evidence preview, and status update.
Runtime infrastructure	Docker, Docker Compose, PostgreSQL, Redis, RabbitMQ, Cloudinary, and Azure App Service configuration.

3.4 System Design Method

The backend was designed as a small microservice system. The service split was kept limited because too many services would make the prototype harder to operate without adding much value. Microservice literature emphasizes that service boundaries should clarify responsibility and support change, while still accounting for operational cost [9, 11, 10]. VietFlood therefore uses only the boundaries needed for the thesis: public access, authentication, report management, shared infrastructure utilities, and supporting infrastructure.

The API gateway receives HTTP and WebSocket traffic from clients. The authentication service owns registration, sign-in, refresh tokens, profile loading, logout, admin seeding, and user management. The reports service owns report creation, report lookup, evidence metadata, status changes, cache updates, and report deletion. Redis supports cached report lookup, RabbitMQ carries service messages, PostgreSQL stores persistent data, and Cloudinary stores uploaded images.

3.4.1 API Gateway Method

The gateway exposes the routes used by both client applications. Authentication routes are grouped under `/auth`, and report routes are grouped under `/reports`. Report creation uses multipart upload through the `files` field. The gateway holds uploaded files in memory long enough to upload them to Cloudinary. It then forwards the report data and evidence metadata to the reports service.

The gateway also applies authentication and role checks. Citizen routes are used for personal report operations. Broader report-reading routes are available to relief and admin

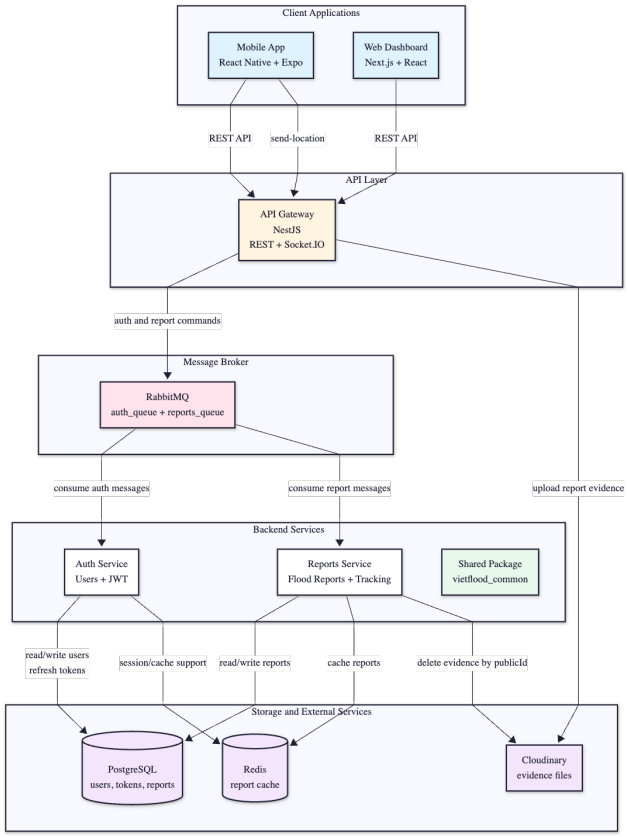

Figure 3.4: VietFlood system architecture diagram.

users. Admin-specific operations are protected separately. This method keeps the clients simple while still enforcing access rules on the backend.

3.4.2 Message Communication Method

RabbitMQ is used for communication between the gateway and internal services [13]. Authentication messages are consumed by the auth service, while report messages are consumed by the reports service. The authentication service handles message patterns for registration, sign-in, profile loading, token refresh, logout, and user operations. The reports service handles report creation, listing, lookup, update, and deletion.

This message-based method makes boundaries visible during development. A failed report submission can be inspected at the gateway request, the Cloudinary upload step, the RabbitMQ message, or the reports service handler. The gateway also uses controlled service-call behavior so a slow internal service does not leave the client waiting without a response.

3.5 Database Design Method

The database design was derived from the report workflow. A report needs a user owner, location fields, status, urgency, severity, evidence metadata, and timestamps. The backend uses PostgreSQL as the main database and TypeORM entities to map application objects to database tables [14, 15].

The user entity stores email, username, phone, hashed password, role, name fields, date of birth, address line, ward, province, and timestamps. Email, username, and phone

are unique. The supported roles are `citizen`, `relief`, and `admin`.

The report entity stores category arrays, description, image URLs, evidence JSON, province, ward, address line, latitude, longitude, status, reporter name, urgency, severity, timestamps, and owner user ID. Evidence is stored as JSON metadata because each uploaded item needs a URL, Cloudinary public ID, and resource type. PostgreSQL supports this kind of flexible structured field through JSON storage [15].

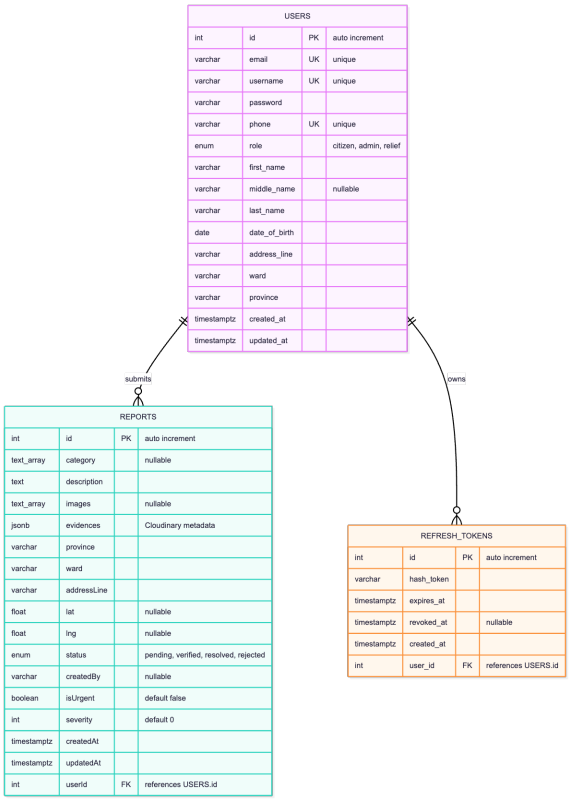

Figure 3.5: Database schema diagram for VietFlood.

3.5.1 Cache Method

Redis is used as a cache and not as the source of truth [16]. When a report is created, the reports service stores a cached copy using a key pattern based on the report ID. When a report is requested, the service can check Redis before reading from PostgreSQL. When the report is updated or deleted, the cache entry is refreshed or removed.

The cache scope is intentionally narrow. Report ownership, status, evidence, and location remain in PostgreSQL. Redis only provides a temporary lookup copy. This keeps invalidation understandable for the prototype.

3.6 Workflow Design Method

The main workflow is citizen report creation. The citizen fills in category, description, administrative location, coordinates, urgency, severity, and optional image evidence. The mobile app sends the report as a multipart request. The API gateway authenticates the user, uploads evidence files to Cloudinary, and forwards the completed payload to the reports service through RabbitMQ. The reports service saves the report in PostgreSQL and creates or updates the Redis cache entry.

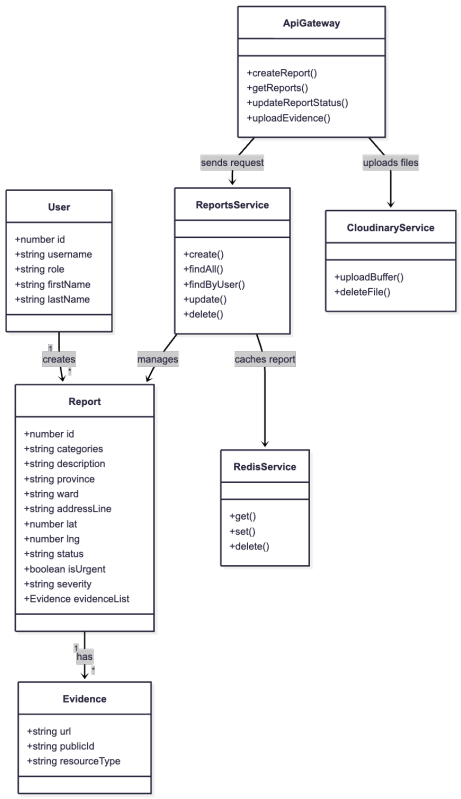

Figure 3.6: Class or module diagram for the report workflow.

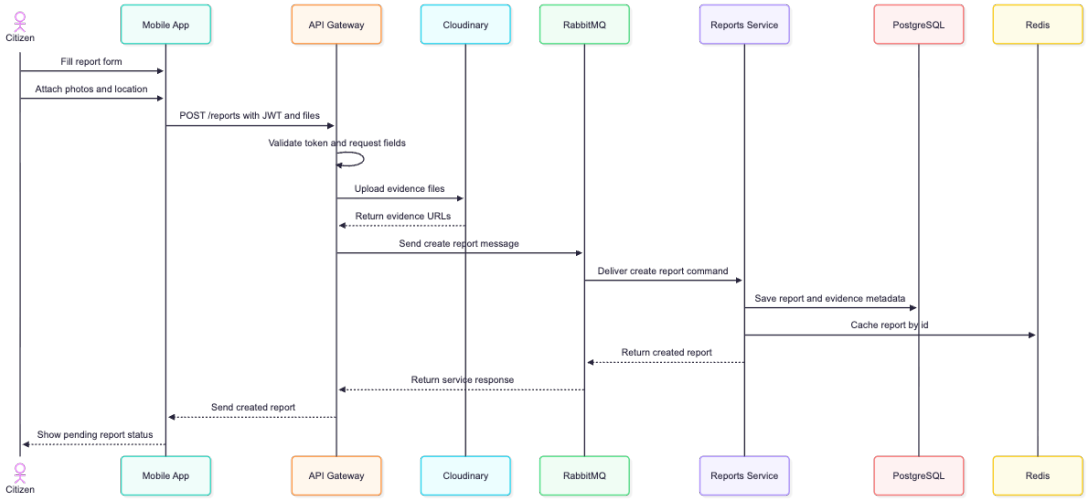

Figure 3.7: Sequence diagram for creating a VietFlood report.

3.6.1 Evidence Handling Method

Evidence images are not stored in PostgreSQL as raw files. The gateway uploads image buffers to Cloudinary under the report folder [8]. The reports service stores the returned URL, public ID, and resource type with the report. The URL is used by the mobile app and dashboard for display. The public ID is kept so related media can be removed when a report is deleted.

This method keeps the database focused on structured data while Cloudinary handles media storage and delivery. It also means media credentials must be provided through environment configuration instead of being written into the source code.

3.6.2 Status Review Method

Status review is handled as an administrator workflow. Relief users can read broader report information and use map-related screens, but verification and rejection are not part of the relief role in this thesis. The administrator dashboard sends status updates through the gateway. The reports service checks the existing report, merges new evidence metadata when needed, updates the status, and refreshes the cache entry.

3.7 Client Application Method

3.7.1 Mobile Application Method

The mobile app was developed with Expo React Native and TypeScript [5, 6, 4]. It supports token-based authentication, profile screens, report creation, report lists, relief report details, map screens, settings, localization, and role-aware navigation. Redux Toolkit supports application state around authentication and client-side data flow [29].

The mobile report service normalizes backend data before rendering it. This is needed because report fields may appear in different naming styles, such as snake case and camel case, or as `lat`/`lng` and `latitude`/`longitude`. The mobile location method also combines administrative division data from the Vietnam Provinces Open API with device location when permission is available [7].

3.7.2 Web Dashboard Method

The web dashboard was developed with Next.js, React, TypeScript, and Tailwind CSS [21, 22, 4, 23]. It is focused on administrator review. The dashboard signs in through the backend, stores access and refresh tokens, refreshes sessions when needed, loads the current profile, and blocks non-admin users from the admin workspace.

After authentication, the dashboard loads report data from the gateway and displays report cards with status, category, severity, address, description, reporter information, and evidence thumbnails. It also provides search, filtering, user management, and status update controls.

3.8 Real-Time Tracking Method

The backend includes a Socket.IO gateway for live location updates [17]. A connected client can emit a location event with latitude, longitude, accuracy, heading, speed, and timestamp. The gateway checks latitude and longitude ranges before broadcasting the

accepted location to other clients. Invalid payloads receive an error event. When a socket disconnects, the gateway announces the disconnected client.

This feature is treated as a prototype path. It proves that the backend can receive and broadcast live location data. Future work would still be needed for authenticated socket sessions, route assignment, persistent location history, and operational use in real relief coordination.

3.9 Validation Method

Validation was performed through source-level checks and workflow checks. Source-level checks focused on whether each module matched its responsibility: the gateway handled client-facing access, the authentication service handled identity and tokens, the reports service handled report data, and the shared package handled reusable infrastructure. Docker Compose was used as the repeatable local environment for backend services and infrastructure dependencies [18].

Workflow checks focused on the main paths of the prototype:

- citizen registration, sign-in, and profile loading;
- citizen report creation with category, address, coordinates, urgency, severity, and images;
- evidence upload to Cloudinary and metadata storage in PostgreSQL;
- report list loading for citizen, relief, and admin views;
- administrator report status update through the web dashboard;
- Redis cache creation, refresh, and deletion during report changes;
- Docker Compose startup for backend services, Redis, and RabbitMQ.

This validation method matches the current stage of VietFlood. The system is a working prototype, not a certified emergency-response platform. The checks therefore focus on whether the main reporting, evidence, review, cache, and service-integration flows operate according to the design.

Chapter 4
Implementation and Result

This chapter describes how VietFlood was implemented as a working prototype. The aim was to keep the system small enough to run and test locally, while still separating the main responsibilities: public API handling, user authentication, report processing, evidence storage, caching, and administrator review.

The implementation is organized into four deliverables. The backend consists of an API gateway, an authentication service, and a reports service. The mobile application supports citizen reporting and relief viewing. The web dashboard supports administrator review. A shared package, `vietflood_common`, provides common infrastructure utilities used by the backend services.

4.1 Technique and Tools

VietFlood uses TypeScript across the backend and clients to reduce context switching and to keep data models consistent. The backend is built with NestJS [3] and runs as a small set of services communicating through a message broker [12]. The mobile application is built with Expo React Native [5, 6]. The web dashboard is built with Next.js [21].

The project is developed and deployed as containers. Docker Compose is used to run the gateway, services, RabbitMQ, and Redis together for local development [18]. For deployment, images are built and deployed to Azure App Service [19]. Secret values (database URL, JWT secrets, Cloudinary credentials, and broker credentials) are provided through environment variables rather than stored in source code.

Table 4.1: Main implementation tools used in VietFlood.

Area	**Tools and implementation use**
Backend	NestJS, TypeScript, TypeORM, RabbitMQ, Socket.IO, JWT, bcrypt, and Docker.
Shared library	`vietflood_common` for logging, Redis access, and Cloudinary operations.
Database and cache	PostgreSQL for persistent data and Redis for report cache entries.
Media storage	Cloudinary for report evidence and generated media URLs.
Mobile app	Expo, React Native, React Navigation, Redux Toolkit, SecureStore, image picker, and device location.
Web dashboard	Next.js, React, TypeScript, Tailwind CSS, token storage, and admin report review.
Deployment	Docker Compose, Docker Hub, GitHub Actions, and Azure App Service.

4.2 Messaging Broker Implementation

The API gateway is the only backend component exposed to the clients. It accepts HTTP requests (including multipart uploads) and forwards work to internal services using RabbitMQ [13, 12]. This keeps the gateway focused on transport-level concerns (routing, guards, role checks, and upload parsing), while the auth service and reports service hold the business logic and database operations.

In practice, the gateway sends message patterns for actions such as registration, sign-in, profile loading, refresh token rotation, report creation, report listing, report updates, and report deletion. Each internal service listens to its own queue and returns a response payload back to the gateway. The gateway applies timeouts and retries, with longer limits on paths that include evidence upload to Cloudinary.

Table 4.2: Comparison of message brokers for the VietFlood backend.

Criteria	**RabbitMQ**	**Redis Pub/Sub**	**Kafka**
Request-response work	Fits command-style gateway requests and service responses.	Lightweight broadcast, but not designed for reliable commands.	Strong streaming model, but unnecessary complexity for this prototype.
Operational cost	Easy to run with Docker Compose alongside the services.	Already required as a cache in VietFlood, but Pub/Sub lacks queue semantics.	Requires more configuration, topic design, and operational overhead.
VietFlood usage	Used for auth and reports service communication.	Used separately as the report cache layer.	Not used in the current system.

Figure 4.1: RabbitMQ communication flow between the API gateway, auth service, reports service, auth queue, and reports queue.

4.3 Authentication and Authorization Implementation

VietFlood uses password-based sign-in and issues JWT access tokens and refresh tokens. Passwords are hashed with bcrypt before storage [25]. Access tokens follow the JSON Web Token format [26] and include the user identity and role information required by the gateway guards.

4.3.1 Authentication Mechanism Implementation

a) Registration Process

Registration is handled by the auth service. The service checks whether `email`, `username`, or `phone` are already used. When the input is valid, the password is hashed and a new row is inserted into the `users` table. The response confirms registration without returning any sensitive fields.

b) Login Process

Login uses `POST /auth/sign_in`. The auth service looks up the user by username and compares the provided password with the stored bcrypt hash. If validation succeeds, the service returns an access token and a refresh token. The mobile application supports either persistent sign-in (SecureStore) or session-only sign-in, and token refresh follows the same storage choice.

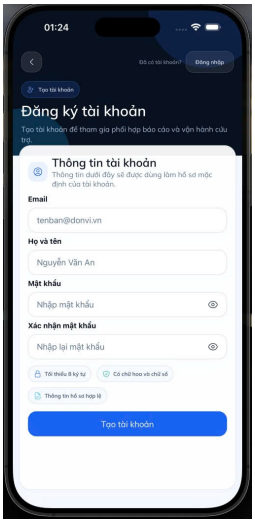

Figure 4.2: Citizen registration screen in the VietFlood mobile app.

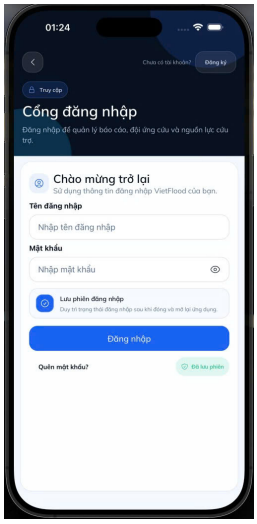

Figure 4.3: Citizen login screen in the VietFlood mobile app.

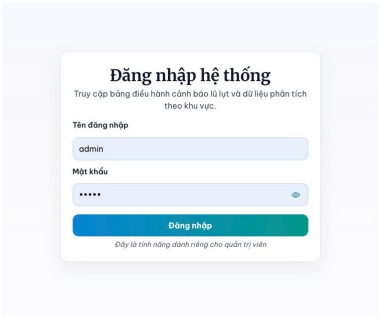

Figure 4.4: Admin login page in the VietFlood web dashboard.

4.3.2 Refresh Token Implementation

Refresh tokens are stored as hashed values in the `refresh_tokens` table and include expiration and revocation fields. The refresh endpoint supports issuing a new access token without requiring a full login. In the prototype, refresh token rotation helps the client remain signed in while still allowing sessions to be ended through token revocation logic.

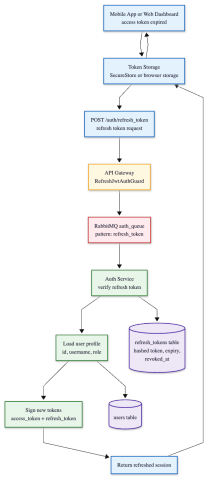

Figure 4.5: Refresh-token flow from mobile or web client through the API gateway, auth service, refresh-token table, and new access token response.

4.3.3 Authorization Process Implementation

Authorization is enforced at the API gateway. JWT guards validate access tokens, while role guards limit routes to citizen, relief, or administrator roles. Citizens can create and manage their own reports. Relief users focus on reading and location-aware viewing. Administrator routes include status changes and user management. In this prototype, verification and rejection remain administrator actions to keep the review decision under one role.

4.4 Report Processing and Evidence Upload Implementation

Report creation is implemented as a single request that can include both structured fields and evidence images. The mobile client sends multipart form data with report fields such as `description`, administrative address fields, coordinates, category, urgency, and severity, plus an optional set of image files.

The gateway receives the request, uploads evidence files to Cloudinary [8], and builds evidence metadata (URL, public ID, resource type). The final payload is forwarded to the reports service via RabbitMQ. The reports service stores the report in PostgreSQL and stores evidence metadata as JSON fields [15]. After saving, it writes a Redis cache entry for report lookup [16].

4.4.1 Report Update and Delete Implementation

Report update follows the same upload and merge pattern as report creation. When a user attaches new evidence, the gateway uploads the new files to Cloudinary and appends the resulting metadata. The reports service checks ownership, merges new evidence with existing evidence, updates the derived `images` list, and refreshes the Redis cache entry.

Report deletion also checks ownership. When a report is deleted, the service reads Cloudinary public IDs from stored evidence metadata and removes the evidence before deleting the database row. The cache entry for the report is removed to avoid stale reads.

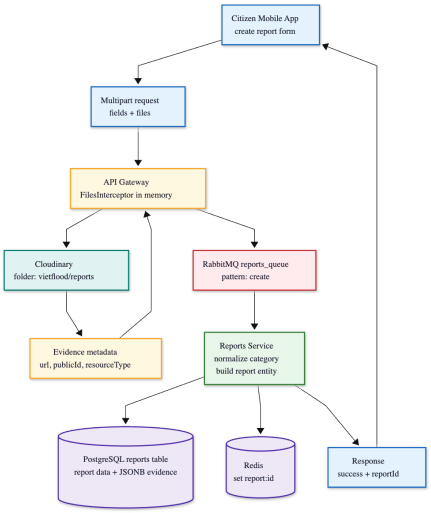

Figure 4.6: Report creation flow from mobile form submission to Cloudinary upload, RabbitMQ message, PostgreSQL insert, and Redis cache write.

4.5 Public Library Implementation

The shared package `vietflood_common` keeps backend infrastructure concerns in one place. It provides a logger used across services, a Redis helper for common cache operations, and a Cloudinary helper for upload and deletion operations [16, 8]. Centralizing these utilities makes the services smaller and reduces repeated code when adding new features.

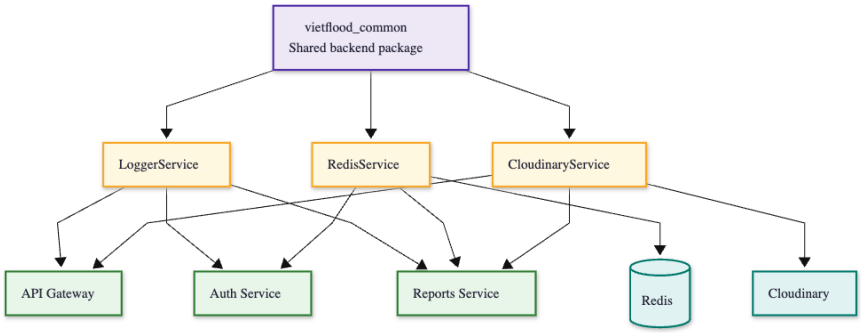

Figure 4.7: Shared common library structure showing LoggerService, RedisService, CloudinaryService, and the backend modules that import them.

4.6 Real-Time Tracking Implementation

VietFlood includes a Socket.IO gateway for live location sharing [17]. The client emits `send-location` events containing latitude, longitude, and optional motion fields such as accuracy, heading, speed, and timestamp. The gateway validates coordinate ranges before broadcasting the location to connected clients. Invalid payloads result in a `location-error` message returned to the sender.

The tracking gateway also emits a disconnect event so other clients can react when a socket leaves. The feature is implemented as a working real-time communication path in the prototype. Authentication of sockets and persistent history storage can be added later if live tracking becomes part of an operational workflow.

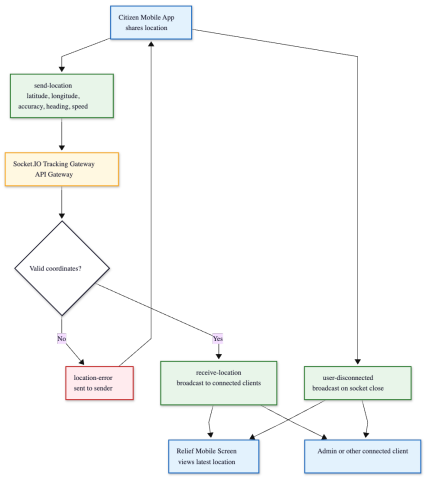

Figure 4.8: Socket.IO live tracking flow with send-location, receive-location, location-error, and user-disconnected events.

4.7 Mobile Citizen Feature Implementation

4.7.1 Report List and Profile Access

After sign-in, the mobile app loads the current user profile through a protected endpoint and displays basic account data. The citizen report list is loaded through the gateway and shows report status, category, and key location fields. During development, the mobile app includes a small normalization layer for fields that changed naming between client versions.

4.7.2 Create Report Implementation

The create report screen collects category, description, address fields, coordinates, urgency and severity, and evidence images. Province and ward selection uses the Vietnam Provinces Open API to reduce spelling variation and improve consistency [7]. When the user grants location permission, the app can prefill coordinates from the device.

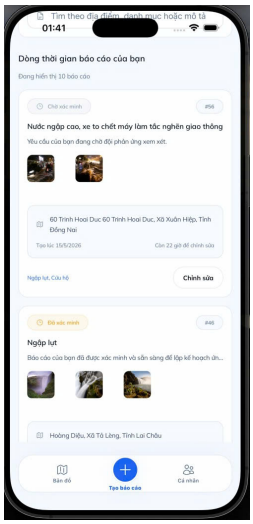

Figure 4.9: Citizen report list or citizen home screen showing submitted reports, status labels, and report location information.

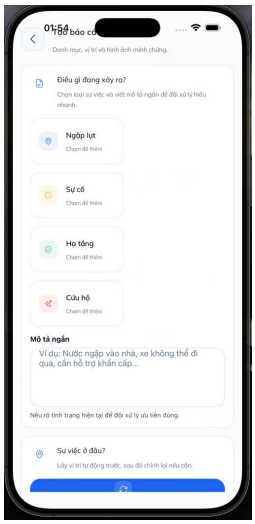

Figure 4.10: Create report form in the VietFlood mobile app.

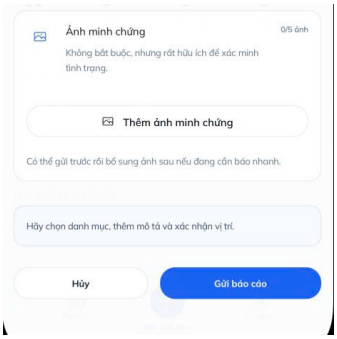

Figure 4.11: Evidence upload in the VietFlood mobile app.

4.8 Relief Feature Implementation

Relief features in the mobile app focus on reading and locating reports. The relief report list provides a fast overview of submitted cases, while the detail screen shows description, category, address fields, evidence, and status. A map view supports location awareness when multiple reports arrive across different areas.

The relief role is intentionally narrower than the administrator role. Relief users can read report information, but verification, rejection, and resolution remain administrator actions in this prototype.

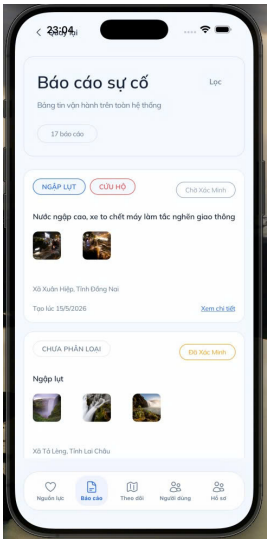

Figure 4.12: Relief report list in the mobile app.

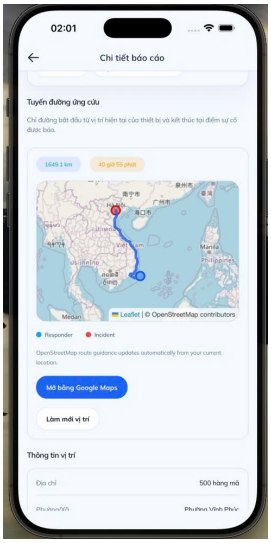

Figure 4.13: Relief map screen in the mobile app.

4.9 Web Dashboard Implementation

The web dashboard is the administrator workspace. After login, it stores tokens, loads the current profile, and blocks access when the user does not satisfy role requirements. The dashboard loads report records through the API gateway, formats report content for review, and presents evidence thumbnails when available.

The interface supports status filtering and basic search to help administrators navigate many incoming reports. When an administrator updates a report status, the dashboard sends the change through the gateway, which forwards the update request to the reports service through RabbitMQ.

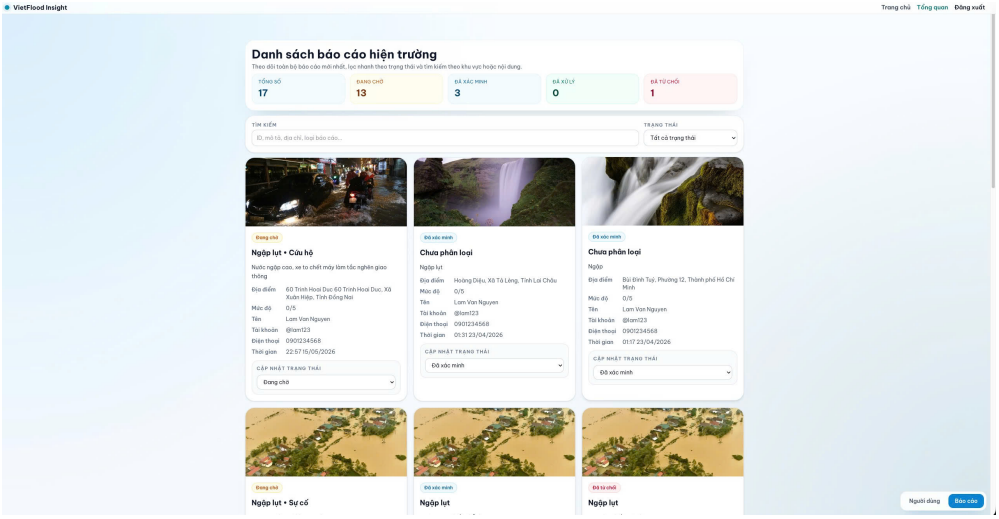

Figure 4.14: Admin dashboard overview with report statistics, search, status filters, report cards, and evidence thumbnails.

4.9.1 Evidence Review and Status Update

Evidence review uses Cloudinary URLs stored on the report. The dashboard prefers the evidence metadata list when present, and can fall back to an image URL list for backward compatibility with earlier client payloads. Status updates are performed through protected routes and trigger cache invalidation in the reports service so subsequent reads reflect the new state.

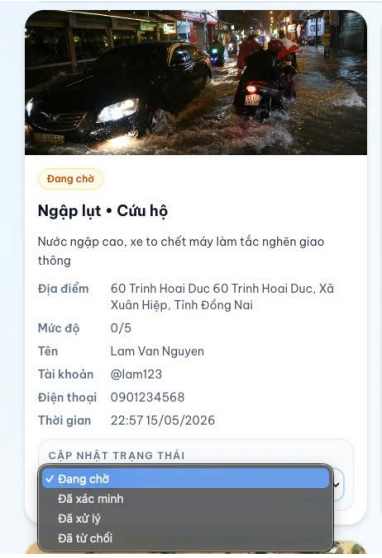

Figure 4.15: Admin evidence review and status update screen for pending, verified, resolved, and rejected reports.

4.10 Deployment Implementation

The backend is packaged and deployed as Docker images. In local development, Docker Compose builds and runs the API gateway, auth service, and reports service alongside RabbitMQ and Redis [18]. In a deployment configuration, the same services can be pulled as prebuilt images and run with environment variables supplied by the platform [19].

The automated workflow builds images, pushes them to Docker Hub, and updates the Azure App Service container configuration using GitHub Actions [20]. This approach reduces manual deployment steps and aligns with continuous delivery practices [27, 28].

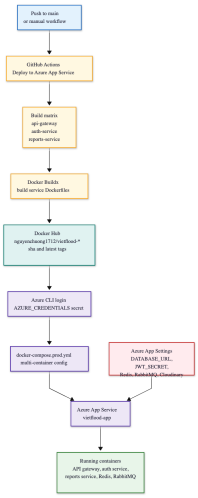

Figure 4.16: CI/CD workflow from GitHub push to Docker image build, Docker Hub push, Azure App Service configuration, and application restart.

4.11 User Management Implementation

User management is implemented in the auth service and exposed through the API gateway. The service supports listing users, fetching a user by id, updating profiles, and

deleting users. Roles are stored as a database field and are used by the gateway guards to enforce access boundaries.

On the client side, the web dashboard validates the current profile before showing administrator-only screens. The mobile application uses protected profile and update endpoints to allow users to view and update their own information. The role model remains deliberately simple in the prototype, but it is sufficient to separate citizen reporting, relief viewing, and administrator review.

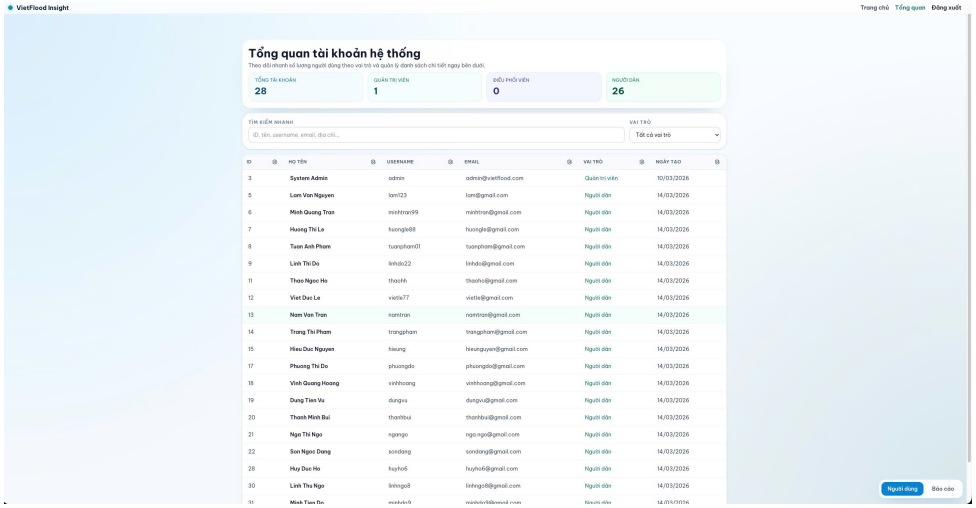

Figure 4.17: User management view showing citizen, relief, and admin accounts with profile and role information.

Chapter 5
Evaluation

This chapter evaluates the VietFlood prototype as an implemented flood reporting system. The evaluation does not attempt to measure real-world impact in a live flood response operation. Instead, it checks whether the delivered backend services and client applications support the intended workflow from Chapter 4: users can authenticate, submit reports with location and evidence, store and retrieve reports, and review them through role-based interfaces.

The evaluation uses scenario walkthroughs and source-level inspection of the API gateway, authentication service, reports service, mobile application, web dashboard, and shared library. A complete list of REST endpoints and representative responses are provided in Appendix A (Table A.1). The system-level test cases used as a checklist for this evaluation are listed in Appendix C (Tables C.1–C.3). The results here should be read as prototype validation rather than production certification.

5.1 Evaluation Scope and Method

The evaluation is structured around the three user roles supported by VietFlood:

Citizen: register and sign in, create flood reports with location and evidence, and view personal report history on mobile.

Relief user: browse report information and location-oriented views on mobile.

Administrator: review reports, inspect evidence, change report status, and manage users on the web dashboard.

On the backend, the API gateway acts as the public interface for HTTP routes and Socket.IO events [17]. Authentication and profile logic are handled by the auth service. Report creation, update, deletion, and cache handling are handled by the reports service. The gateway communicates with internal services through RabbitMQ message patterns [13]. This separation follows the goal of defining clear service responsibilities while keeping the system manageable as a prototype [24].

5.2 Functional Coverage

The central functional requirement is the end-to-end report workflow. A citizen submits a report from the mobile client. The gateway receives multipart form data, uploads evidence files to Cloudinary [8], and forwards the final payload to the reports service through RabbitMQ [13]. The reports service persists the report in PostgreSQL and stores evidence metadata as JSON fields [15]. After creation and update operations, a Redis cache entry is written to support subsequent lookups [16].

On the review side, administrators use the web dashboard to list reports, inspect evidence thumbnails, and update status values such as `pending`, `verified`, `resolved`, and `rejected`. Relief users can read report information through role-restricted endpoints and present it in list and map oriented views on the mobile application. Together, these features form the expected loop: submission, storage, review, and status update.

Table 5.1: Evaluation summary of key VietFlood workflows.

Workflow	Primary interface	Observed behavior in prototype
Citizen sign in and profile access	`/auth/sign_in`, `/auth/profile`	Tokens are issued after password validation; protected profile data is returned when a JWT is provided.
Create report with evidence	`/reports/create` (multipart)	Evidence files are uploaded to Cloudinary and stored as metadata; the report is persisted in PostgreSQL and cached in Redis.
View citizen report history	`/reports/user`	The signed-in citizen can list reports associated with the current account.
Admin review and status update	Web dashboard + protected report routes	Administrators can list reports, inspect evidence, and update report status values through the gateway.
Live tracking broadcast	Socket.IO: `send-location`	Valid coordinate payloads are broadcast to connected clients; invalid payloads return an error event.

5.3 Client Evaluation

The mobile application is the main field interface and is implemented with Expo React Native [5, 6]. It supports citizen sign-in, profile viewing, report creation with evidence selection, and report history viewing. Relief features focus on reading and mapping report information rather than creating new reports, which matches the intended division of responsibilities between citizen reporting and relief awareness.

The web dashboard is implemented with Next.js [21] and is evaluated as an administrative workspace. Its main functions are report review, evidence inspection, and status updates. It also loads the current profile and blocks dashboard access when a user does not satisfy role checks.

One weakness observed in the client layer is data contract consistency. The mobile code includes normalization logic for location fields such as `lat`/`lng` and `latitude`/`longitude`. This improves tolerance during development, but it indicates that the API contract should be stabilized before broader use, with consistent DTO validation and a single naming convention for coordinates and address fields.

5.4 Data Integrity and Storage

Evidence files are stored in Cloudinary, while PostgreSQL stores report data and evidence metadata. This keeps the database focused on structured fields and avoids storing raw media as database blobs [8]. Evidence metadata is stored in JSON fields (URL, public ID, resource type), which is a reasonable prototype choice when the evidence structure may evolve [15].

The Redis cache is used conservatively. A report cache entry is written after create and update operations, and it is cleared and refreshed when a report changes or is deleted [16]. This design reduces repeated reads for individual report lookups, but it does not optimize large list queries. If VietFlood expands to heavier map queries, province filtering, or large result sets, caching and indexing should be extended beyond single report keys, and pagination should be enforced across list endpoints.

5.5 Security and Access Control

Route protection is enforced at the API gateway through JWT authentication and role guards. Passwords are stored using bcrypt hashing [25]. JWT access tokens carry identity and role claims used for authorization decisions [26]. Refresh tokens are issued using a

separate secret and expiration policy and are intended to support session control and logout.

Several security concerns remain prototype-level. Refresh token rotation and revocation should be tested under concurrent refresh requests and replay attempts. Socket.IO tracking currently validates coordinate ranges, but the tracking channel should be tied to authenticated identities and a clearer authorization model. Evidence upload should also enforce stricter limits on file size, type, and count to reduce abuse and unexpected storage cost.

5.6 Deployment and Operational Readiness

Local deployment uses Docker Compose to run the API gateway, services, RabbitMQ, and Redis in a repeatable environment [18]. The prototype deployment workflow builds Docker images and deploys them to Azure App Service through GitHub Actions [20, 19]. Runtime configuration uses environment variables for database connectivity, RabbitMQ, Redis, JWT secrets, refresh token secret, Cloudinary credentials, and admin seed settings.

The main operational gap is observability. The shared logger provides a consistent logging interface, but a more reliable deployment would also include explicit health endpoints, service-level metrics, and alerting. These are important for detecting failures such as evidence upload errors, queue issues, or service unavailability without manual inspection.

5.7 Evaluation Summary

The evaluation shows that VietFlood meets the core prototype requirements. Citizens can submit reports with location and evidence; the backend stores reports and evidence metadata; administrators can review reports and change status; and relief users can browse report information with location context. The main gaps are hardening tasks needed for real deployment: a stable API contract, systematic tests under adverse network conditions, better upload controls, stronger tracking authorization, and monitoring suitable for unattended operation. Within the thesis scope, VietFlood demonstrates that a microservice-based design using NestJS microservice support [12] can provide a practical foundation for multi-platform flood situation reporting.

Chapter 6
Conclusion and Future Works

6.1 Conclusion

This thesis developed VietFlood, a multi-platform prototype for flood situation reporting and review. The core idea is simple: flood information often appears first as local observations, but that information is hard to search, verify, and follow when it lives only inside chat threads and scattered photos. VietFlood turns those observations into structured reports with location fields and evidence links so that the information can be stored, reviewed, and updated over time.

The final prototype combines a NestJS backend (API gateway, auth service, and reports service), an Expo React Native mobile app for citizen and relief workflows, and a Next.js web dashboard for administrator review. Evidence files are stored in Cloudinary and referenced by metadata in the report record. Redis and RabbitMQ support caching and service-to-service communication, and the gateway enforces JWT authentication and role checks. This architecture keeps the responsibilities separated while still being practical to run and test as a small prototype [9, 11, 10].

Most importantly, the system demonstrates the full reporting loop end-to-end: a citizen can sign in, submit a report with location and optional evidence, the backend can store it reliably, and an administrator can review the evidence and update the report status. Relief users can browse report information and location-aware views without taking over the verification role. That role split is intentional: it keeps status changes under administrator control, which matters because status is the main trust signal shown back to users.

VietFlood is still a prototype. It has not been validated under real flood conditions, large traffic, or unreliable connectivity. The system works as a technical foundation, but it needs deeper testing and hardening before it could be used in an operational setting.

6.2 Future Work

Future work should start with real usage feedback. The most valuable questions are not about technology choices, but about whether the reporting flow fits how people act during a flood: how fast they can submit a report, whether they understand the address fields, and how much evidence is realistic to upload from a phone under weak networks.

The mobile client should be improved for unreliable connectivity. A practical next step is an offline-first report draft model with queued retries and clear states (draft, queued, uploading, submitted, failed). Evidence files should be compressed client-side when possible, and the gateway should enforce upload limits (file count, size, and allowed types) to reduce abuse and unexpected storage cost.

The review workflow can also be improved. Duplicate detection, spam handling, and clustering by time and distance could reduce administrator workload. On the relief side, additional features such as route notes or assignment could help coordination, but the verification decision should remain an administrator responsibility so that status does not drift across roles.

Finally, deployment and security need stronger engineering before any public use. Refresh token rotation and revocation should be tested under edge cases, Socket.IO tracking

should be tied to authenticated identities, and the system should include health checks and monitoring so failures are visible quickly. These improvements are typical operational work and align with DevOps automation and deployment practices [28, 27]. Docker Compose provides a solid base for repeatable environments, but production operation requires observability and recovery procedures beyond a local compose file [18].

Bibliography

[1] M. F. Goodchild and J. A. Glennon, "Crowdsourcing geographic information for disaster response: a research frontier," *International Journal of Digital Earth*, vol. 3, no. 3, pp. 231–241, 2010.

[2] M. Imran, C. Castillo, F. Diaz, and S. Vieweg, "Processing social media messages in mass emergency: A survey," *ACM Computing Surveys*, vol. 47, no. 4, pp. 67:1–67:38, 2015.

[3] NestJS, "Nestjs documentation." `https://docs.nestjs.com/`, 2026. Accessed: 2026-05-17.

[4] TypeScript, "Typescript documentation." `https://www.typescriptlang.org/docs/`, 2026. Accessed: 2026-05-17.

[5] Expo, "Expo documentation." `https://docs.expo.dev/`, 2026. Accessed: 2026-05-17.

[6] Meta Open Source, "React native documentation: Getting started." `https://reactnative.dev/docs/getting-started`, 2026. Accessed: 2026-05-17.

[7] Vietnam Open API, "Vietnam provinces open api." `https://provinces.open-api.vn/`, 2026. Accessed: 2026-05-17.

[8] Cloudinary, "Upload api reference." `https://cloudinary.com/documentation/image_upload_api_reference`, 2026. Accessed: 2026-05-17.

[9] M. Fowler and J. Lewis, "Microservices: A definition of this new architectural term." `https://martinfowler.com/articles/microservices.html`, 2014. Accessed: 2026-05-17.

[10] J. Thönes, "Microservices," *IEEE Software*, vol. 32, no. 1, p. 116, 2015.

[11] N. Dragoni, S. Giallorenzo, A. Lluch Lafuente, M. Mazzara, F. Montesi, R. Mustafin, and L. Safina, "Microservices: Yesterday, today, and tomorrow," in *Present and Ulterior Software Engineering*, pp. 195–216, Springer International Publishing, 2017.

[12] NestJS, "Nestjs microservices documentation." `https://docs.nestjs.com/microservices/basics`, 2026. Accessed: 2026-05-17.

[13] RabbitMQ, "Consumer acknowledgements and publisher confirms." `https://www.rabbitmq.com/docs/3.13/confirms`, 2026. Accessed: 2026-05-17.

[14] TypeORM, "Typeorm documentation: Getting started." `https://typeorm.io/docs/getting-started/`, 2026. Accessed: 2026-05-17.

[15] PostgreSQL Global Development Group, "Postgresql documentation: Json types." `https://www.postgresql.org/docs/current/datatype-json.html`, 2026. Accessed: 2026-05-17.

[16] Redis, "Redis documentation." https://redis.io/docs/latest/, 2026. Accessed: 2026-05-17.

[17] Socket.IO, "Socket.io documentation." https://socket.io/docs/v4/, 2026. Accessed: 2026-05-17.

[18] Docker, "Docker compose documentation." https://docs.docker.com/compose/, 2026. Accessed: 2026-05-17.

[19] Microsoft, "Continuous deployment with custom containers in azure app service." https://learn.microsoft.com/en-us/azure/app-service/deploy-ci-cd-custom-container, 2026. Accessed: 2026-05-17.

[20] GitHub, "Github actions documentation." https://docs.github.com/en/actions/, 2026. Accessed: 2026-05-17.

[21] Vercel, "Next.js documentation." https://nextjs.org/docs, 2026. Accessed: 2026-05-17.

[22] React, "React documentation." https://react.dev/, 2026. Accessed: 2026-05-17.

[23] Tailwind Labs, "Tailwind css documentation." https://tailwindcss.com/docs, 2026. Accessed: 2026-05-17.

[24] L. Bass, P. Clements, and R. Kazman, *Software Architecture in Practice*. Addison-Wesley Professional, 4 ed., 2021.

[25] N. Provos and D. Mazières, "A future-adaptable password scheme," in *1999 USENIX Annual Technical Conference*, (Monterey, California), USENIX Association, June 1999. Accessed: 2026-05-17.

[26] M. B. Jones, J. Bradley, and N. Sakimura, "Json web token (jwt)," RFC 7519, Internet Engineering Task Force, 2015. Accessed: 2026-05-17.

[27] J. Humble and D. Farley, *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*. Addison-Wesley Professional, 2010.

[28] G. Kim, P. Debois, J. Willis, and J. Humble, *The DevOps Handbook: How to Create World-Class Agility, Reliability, and Security in Technology Organizations*. IT Revolution Press, 2016.

[29] Redux Toolkit, "Redux toolkit documentation." https://redux-toolkit.js.org/, 2026. Accessed: 2026-05-17.

Appendix A
API Response Samples

This appendix summarizes the VietFlood API gateway endpoints and provides representative JSON responses. Token strings, user identifiers, and media URLs are masked with placeholders.

A.1 API Endpoint List

Table A.1: VietFlood REST endpoints exposed by the API gateway.

Method	Endpoint	Access	Description
Authentication			
POST	`/auth/register`	Public	Create a new user account.
POST	`/auth/sign_in`	Public	Authenticate and obtain access and refresh tokens.
POST	`/auth/refresh_token`	Refresh token	Issue a new access token and rotate the refresh token.
POST	`/auth/logout`	Public	Logout and revoke refresh token (if provided).
POST	`/auth/refresh`	Public	Refresh helper endpoint used by the gateway (payload passthrough).
GET	`/auth/profile`	JWT	Fetch the current user profile.
PUT	`/auth/update`	JWT	Update the current user's profile.
PUT	`/auth/update/user/:id`	JWT + Role (admin/relief)	Update a user profile by user id.
DELETE	`/auth/delete/:id`	JWT + Role (admin)	Delete a user by user id.
GET	`/auth/all`	JWT + Role (admin/relief)	List all users.
GET	`/auth/relief/users`	JWT + Role (admin/relief)	List users (alias of `/auth/all`).
GET	`/auth/relief/users/:id`	JWT + Role (admin/relief)	Fetch a user by user id.
Reports			
POST	`/reports/create`	JWT	Create a new flood report (supports evidence file upload).
GET	`/reports`	JWT + Role (admin/relief)	Fetch all reports for admin/relief review.
GET	`/reports/user`	JWT + Role (citizen)	Fetch reports created by the current citizen user.
GET	`/reports/:id`	JWT + Role (admin/relief)	Fetch reports for a specific user id.
PUT	`/reports/update/:id`	JWT	Update a report owned by the current user (supports evidence upload).
PUT	`/reports/update/:id/admin/:userId`	JWT + Role (admin)	Update a report for a specific user id (admin workflow).
PUT	`/reports/relief/:id/user/:userId`	JWT + Role (admin/relief)	Update a report for a specific user id (relief workflow).
DELETE	`/reports/:id`	JWT	Delete a report owned by the current user.
DELETE	`/reports/update/:id/admin/:userId`	JWT + Role (admin)	Delete a report for a specific user id (admin workflow).
DELETE	`/reports/relief/:id/user/:userId`	JWT + Role (admin/relief)	Delete a report for a specific user id (relief workflow).

A.2 Sign In

Endpoint: `POST /auth/sign_in`

Response body:

```
{
  "accessToken": "<JWT_ACCESS_TOKEN>",
  "refresh_token": "<JWT_REFRESH_TOKEN>"
}
```

Listing A.1: Sign In Response

A.3 Refresh Access Token

Endpoint: `POST /auth/refresh_token`

Response body:

```
{
  "access_token": "<JWT_ACCESS_TOKEN>",
  "refresh_token": "<JWT_REFRESH_TOKEN>"
}
```

Listing A.2: Refresh Token Response

A.4 User Profile

Endpoint: `GET /auth/profile`

Response body:

```
{
  "id": 7,
  "email": "<user_email>",
  "username": "<username>",
  "phone": "<phone_number>",
  "role": "citizen",
  "first_name": "<first_name>",
  "middle_name": null,
  "last_name": "<last_name>",
  "date_of_birth": "2000-01-01",
  "address_line": "<address_line>",
  "ward": "<ward>",
  "province": "<province>",
  "created_at": "2026-05-17T00:00:00.000Z",
  "updated_at": "2026-05-17T00:00:00.000Z"
}
```

Listing A.3: User Profile Response

A.5 Create Report

Endpoint: `POST /reports/create`

Response body:

```
{
  "success": true,
  "message": "Create report successfully",
  "reportId": 41,
  "category": ["flood"],
  "evidences": [
    {
      "url": "<CLOUDINARY_URL>",
      "publicId": "<CLOUDINARY_PUBLIC_ID>",
      "resourceType": "image"
    }
  ],
  "images": [
    "<CLOUDINARY_URL>"
  ]
}
```

Listing A.4: Create Report Response

Appendix B
Interface Screenshots

Key user interface screens are shown here. These images support the main report without repeating the implementation details.

B.1 Screenshots

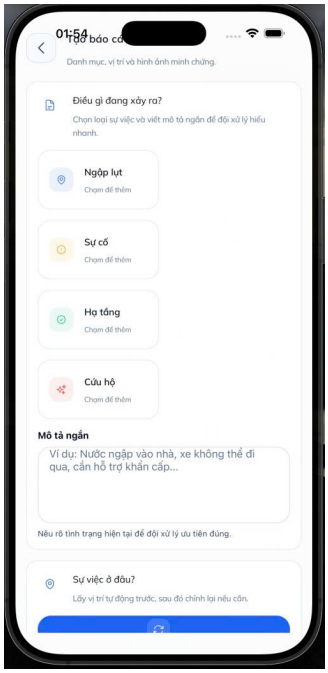

Figure B.1: Citizen report creation screen.

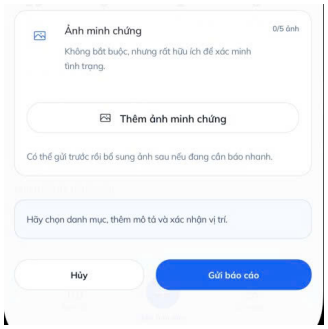

Figure B.2: Evidence upload screen.

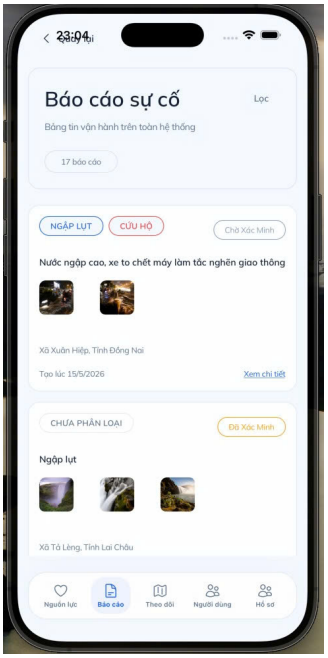

Figure B.3: Relief worker report list screen.

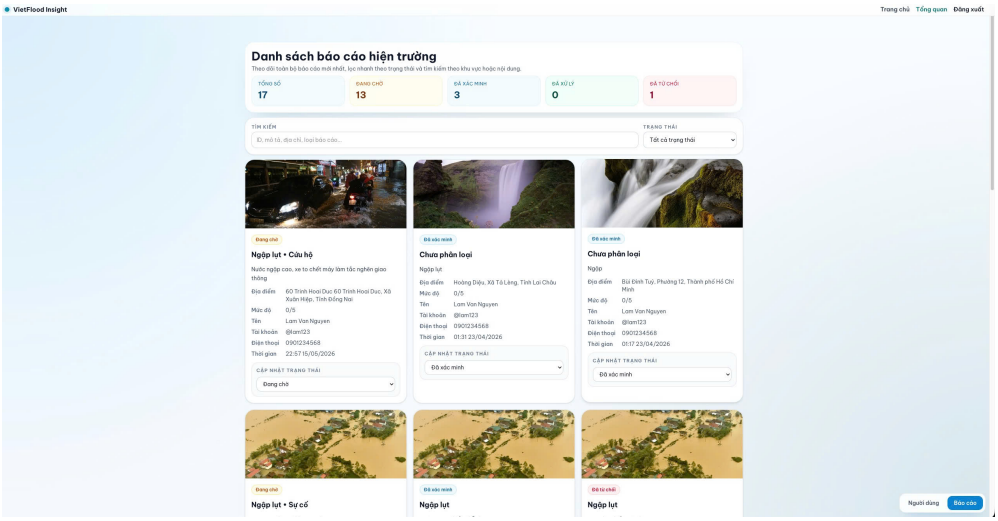

Figure B.4: Admin dashboard overview.

Appendix C
Validation Test Cases

This appendix lists system-level test cases used to validate the VietFlood prototype. The cases focus on end-to-end behavior through the API gateway, including authentication, role checks, report workflow, and evidence handling.

C.1 Authentication and User Management

Table C.1: Authentication and user management test cases.

ID	Feature	Input / Steps (summary)	Expected Result
AUTH-01	Sign in success (`POST /auth/sign_in`)	Provide valid `username` and `password`.	Response contains `accessToken` and `refresh_token`.
AUTH-02	Sign in failure	Provide valid username with incorrect password.	Request rejected with an authentication error (no token returned).
AUTH-03	Access protected route without JWT	Call `GET /auth/profile` without `Authorization: Bearer ...`.	Request rejected.
AUTH-04	Refresh token success (`POST /auth/refresh_token`)	Send `refresh` value in the body containing a valid refresh token.	Response contains `access_token` and a rotated `refresh_token`.
AUTH-05	Role enforcement	Call admin/relief-only routes (for example `GET /reports`) using a citizen JWT.	Request rejected due to insufficient role.
AUTH-06	Admin user listing	Call `GET /auth/all` with an admin JWT.	Returns list of users with profile fields.

C.2 Report Workflow and Evidence Handling

Table C.2: Report workflow test cases.

ID	Feature	Input / Steps (summary)	Expected Result
REP-01	Create report with evidence (`POST /reports/create`)	Submit required fields (`description`, `province`, `ward`, `addressLine`) with `category[]` and 1–3 image files.	Response indicates success and returns `reportId`, `evidences` metadata, and `images[]` URLs.
REP-02	Create report without evidence	Submit report payload with no files.	Report created; `evidences` and `images` are empty arrays.
REP-03	Citizen fetches own reports (`GET /reports/user`)	Call as citizen with valid JWT.	Returns list of reports created by the current user.
REP-04	Admin/relief fetches all reports (`GET /reports`)	Call as admin or relief user.	Returns list of report records for review.
REP-05	Update report by owner (`PUT /reports/update/:id`)	Update an existing report id owned by the current user; optionally attach additional evidence files.	Returns success message and updated evidence list and image URLs.
REP-06	Delete report by owner (`DELETE /reports/:id`)	Delete an existing report id owned by the current user.	Returns success confirmation; subsequent fetch does not include the deleted report.

C.3 Real-time Location Tracking (Socket.IO)

Table C.3: Socket.IO live tracking test cases.

ID	Event	Input / Steps (summary)	Expected Result
TRK-01	Broadcast valid location (`send-location`)	Connect a client and emit `send-location` with valid latitude/longitude ranges.	Server broadcasts `receive-location` containing the sender id and payload.
TRK-02	Reject invalid payload	Emit `send-location` with missing or out-of-range coordinates.	Client receives `location-error` and location is not broadcast.
TRK-03	Disconnect notification	Disconnect a client socket.	Server broadcasts `user-disconnected` with the socket id.

Appendix D
Deployment and Configuration

This chapter summarizes the environment variables used to deploy VietFlood with Docker Compose. Values shown here are variable names and example formats only. Secret values are not included.

Table D.1: Environment variables used to deploy VietFlood with Docker Compose.

Group	**Example Variables**	**Notes**
Gateway	`API_GATEWAY_PORT`, `CORS_ORIGINS`	HTTP port and allowed web origins for the gateway service.
Runtime	`NODE_ENV`	Runtime mode used by the Node.js applications (for example, `production`).
Database	`DATABASE_URL`	PostgreSQL connection string for the deployment database.
Message broker	`RABBITMQ_DEFAULT_USER`, `RABBITMQ_DEFAULT_PASS`, `RABBITMQ_URL`	RabbitMQ credentials and connection URL used by the backend services.
Cache	`REDIS_HOST`, `REDIS_PORT`, `REDIS_PASSWORD`, `REDIS_DB`	Redis connection details used for caching (host, port, password, database index).
Token signing	`JWT_SECRET`, `REFRESH_SECRET`	Secrets used to sign access tokens and refresh tokens.
Admin seed	`ADMIN_EMAIL`, `ADMIN_PASSWORD`	Initial administrator credentials used during seed/startup logic.
Cloudinary	`CLOUDINARY_CLOUD_NAME`, `CLOUDINARY_API_KEY`, `CLOUDINARY_API_SECRET`	Cloudinary credentials used for evidence upload and deletion.

D.1 Deployment Notes

- Use masked values for secrets in the thesis and replace them with actual values in deployment.
- Store credentials in CI/CD secrets or environment configuration, not in source control.
- Change default RabbitMQ credentials before production use.
- Set JWT secrets as cryptographically secure strings.